1) Multiplexing vs Demultiplexing.

Multiplexing	Demultiplexing
Multiplexing is performed at the sender transport layer.	Demultiplexing is performed at the receiver transport layer.
It collects data from multiple application sockets.	It receives segments/datagrams from the network layer.
Each application sends data through its own socket.	The transport layer examines the received transport layer.
The transport layer adds source and destination port numbers	It uses source/destination port numbers to identify the correct socket
The data with the transport header forms a TCP segment or UDP datagram.	IP & port information help identify the destination application.
Thus, data from different applications can share the network connection.	Finally, the data is delivered to the appropriate application socket.

Multiplexing

Applications
↓
sockets
↓
Transport layer
↓
segments.

Demultiplexing

segments
↓
Transport layer
↓
Port/IP information
↓
Correct socket.

Comparative Analysis

C. Sravanthi
192525463

TCP Sockets VS. UDP Datagrams.

2)

TCP sockets	UDP Datagrams.
TCP is a connection-oriented transport protocol	UDP is a connectless transport protocol.
A connection is established before data transfer using the three-way handshake.	No connection establishment is required before sending data.
TCP maintains the state of the connection between sender & receiver.	UDP doesn't maintain TCP-style connection state.
TCP provides a reliable and ordered byte stream	UDP provides message-oriented datagrams without guaranteed delivery or ordering
Lost data can be detected and retransmit.	UDP does not provide retransmission of lost datagrams.
TCP provides flow control and congestion control.	UDP has much lower transport overhead and doesn't provide TCP-style flow/congestion.
TCP is suitable when reliable is important	UDP is useful when low overhead and fast transmision are important.
Flow: Connection → Reliable byte stream → ordered delivery	Flow: Message → UDP datagram → Best-effort delivery

3)

Multicast	Unicast / Broadcast
Multicast is a one to many communication method.	Unicast is one to one, while broadcast is one-to-all within the broadcast domain.
A sender sends a single UDP stream to a specific multicast group.	Unicast may require seperate transmissions for different receivers.
Only hosts that have to joined the multicast group receive the traffic	Broadcast traffic is delivered to all applicable hosts in the broadcast domain.
Network devices replicable packets only where required to reach group members	Repeat unicast or broadcast can create additional network traffic.
If is efficient when the same data must reach many receivers.	Unicast can consume more bandwidth when many receivers need the same data.
It is useful for applications such as group streaming & live distribution.	Unicast is suitable for individual communication, while broadcast is used for reaching all hosts in local domain.
Flow: One sender ↓ Multicast group ↓ selected receivers	Flow: sender ↓ Individual / all broadcast-domain hosts.

TCP congestion control	Traffic Management
TCP congestion control is mainly an end-to-end mechanism.	Traffic management is mainly performed by network devices such as routers.
Tcp observes network conditions through acknowledgements, packet loss & signals.	Routers manage traffic using queues, scheduling and traffic policies.
Slow start initially increases the congestion window rapidly.	Qos can classify and prioritize different types of traffic.
AIMD increases the sending rate gradually and decreases it when congestion is detected.	RED can dectect growing queues and drop/mark packets before severe occurs.
When packets loss indicates possible congestion TCP reduces its transmission drate.	policing and shaping can regulate the amount or rate of traffic network.
The main purpose is to prevent the sender from overwhelming the network.	The main purpose is to control, prioritize and manage traffic within network.
Flows: packet loss/congestion ↓ Tcp detects it ↓ Sending rate decreases.	Flows: Traffic ↓ Router queue/policy ↓ Scheduling, marking, dropping.

M/M/1 - single server	M/M/c - Multi server.
M/M/1 has one server.	M/M/C has parallel servers.
Arrivals are assumed to follows poission process.	It uses the same basic poission arrival and exponential service assumption
Service times are exponentially distributed	serve times are exponentially distributed
Arrival time is represented by λ and single service rate by μ.	Arrival rate is λ, each server has service rate μ.
Traffic intensity is $\rho = \lambda/\mu$.	Traffic intensity per server is $\rho = \lambda/(C\mu)$
For stability, $\rho < 1$	For stability, $\rho < 1$
Average waiting time is queue is: $W_q = \lambda / [\mu(\mu - \lambda)]$	Waiting time is lower because several servers can severe, uses Erlang-C model.
Average time is system: $W = 1/(\mu - \lambda)$.	The pooled capacity is $C\mu$, allowing more customers to be served.
Structure: One queue ↓ One server.	Structure: one common queue ↓ Multiple servers.

6)

priority Queues	Network of Queues.
packets are served according to priority.	Multiple queues are connected together.
High priority packets are served first.	output of one queue becomes input to another
It reduces delay for important traffic	It represents traffic moving thrugh multiple service nodes.
Example: voice traffic gets higher priority than background data.	Example: Packet passing through several routers.
Flow: packets ↓ priority ↓ Scheduler ↓ Transmission	Flow: Queue 1 ↓ Queue 2 ↓ Queue 3 ↓ output.

7)

Quic protocol	TLS 1.3
Quic is a transport protocol over UDP.	TLS 1.3 is a security protocol.
Supports multiple independent streams.	Provides encryption and authentication.
Reduces head-of-line blocking between streams.	Does not provide transport-level multiplexing.
Integrates TLS 1.3 for security.	provides security for Quic and other protocols.
Flow: Application ↓ QUIC ↓ UDP ↓ IP	flow: Application ↓ TLS1.3 ↓ TCP ↓ IP